360 INTERNET SECURITY CENTER

The Linux XOR DDoS Botnets

Network Security Research Lab
Qihoo 360 Technology Co. Ltd.

SECURE 2015

Agenda

360 INTERNET SECURITY CENTER

- Sample analysis
- C&C server statistics
- The C&C protocol
- Attack statistics

LDX: Linux XOR DDoS

360 INTERNET SECURITY CENTER

- A notorious CHINA DDoS bot well documented by ***Malware Must Die***! /***FireEye*** /***Cisco*** /***Level3*** /***…***
- It is written in C++, and only targets Linux platform
  - Mainly for x86 CPU
- An XOR encryption algorithm is heavily used in both sample and C&C communication
- Supported attack types: syn/dns flood
  - tcp_ack_flood was added later

The XOR Encryption Algorithm

360 INTERNET SECURITY CENTER

- Cipher text is produced by XORing plaintext against a 16-byte key
  - 2 keys were found, but only one was widely used

```
84 def XorDecrypt(data, key="BB2FA36AAA9541F0"):
85     plain=""
86     idx = 0
87     while idx < len(data):
88         ch = struct.unpack("B", data[idx])[0]
89         k = struct.unpack("B", key[idx % 0x10])[0]
90         ch ^= k
91         plain += struct.pack("B", ch)
92         idx += 1
93     return plain.split("\x00")[0]
```

XOR Encrypted Sample Configs

360 INTERNET SECURITY CENTER

- C2 list

  e.g., www.gggatat456.com:3507|www.xxxatat456.com:3507

- Shell commands

```
0x80CD800, "cat resolv.conf
0x80CD814, "sh
0x80CD828, "bash
0x80CD83C, "su
0x80CD850, "ps -ef
0x80CD864, "ls
0x80CD878, "ls -la
0x80CD88C, "top
0x80CD8A0, "netstat -an
0x80CD8B4, "netstat -antop
0x80CD8C8, "grep "A"
0x80CD8DC, "sleep 1
0x80CD8F0, "cd /etc
0x80CD904, "echo "find"
0x80CD918, "ifconfig eth0
0x80CD92C, "ifconfig
0x80CD940, "route -n
0x80CD954, "gnome-terminal
0x80CD968, "id
0x80CD97C, "who
0x80CD990, "whoami
0x80CD9A4, "pwd
0x80CD9B8, "uptime
```

XOR Used in C&C Messages

360 INTERNET SECURITY CENTER

- All C&C messages are XOR encrypted
  - Not including the message header

```
[liuya@xcd-desk ~]$hexdump -C register_body.bin
00000000  ef 01 00 00 00 00 00 00  ff ff ff ff 32 2e 36 2e  |............2.6.|
00000010  33 30 2d 72 63 35 2d 63  75 73 74 6f 6d 00 00 00  |30-rc5-custom...|
00000020  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000040  00 00 00 00 00 00 00 00  00 00 00 00 00 69 36 38  |.............i68|
00000050  36 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |6...............|
00000060  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000080  00 00 00 00 00 00 00 00  00 00 00 00 00 00 31 2a  |..............1*|
00000090  33 33 39 32 00 00 00 00  00 00 00 00 00 00 00 00  |3392............|
000000a0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
000000c0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 6c  |...............l|
000000d0  75 74 70 67 75 6c 6c 79  7a 79 61 64 66 6e 65 7a  |utpgullyzyadfnez|
000000e0  70 67 62 79 72 68 76 73  71 77 70 69 64 7a 76 00  |pgbyrhvsqwpidzv.|
000000f0  53 54 41 54 49 43 00 00  00 00 00 00 00 00 00 00  |STATIC..........|
00000100  31 2e 31 2e 35 00 00 00  00 00 00 00 00 00 00 00  |1.1.5...........|
00000110
```

plain register body

```
00000000  36 a0 26 b8 10 01 00 00  00 00 00 00 00 00 00 00  6.&.....  ........
00000010  00 00 00 00 00 00 00 00  00 00 00 00             ........  ....
0000001C  a2 45 32 46 a9 30 36 41  be be c6 ca 07 1f 77 01  .E2F.06A  ......w.
0000002C  6c 72 1f 77 73 1e 51 24  2f 24 4b 5c 57 31 46 30  lr.ws.Q$  /$K\W1F0
0000003C  42 42 32 46 41 33 36 41  41 41 39 35 34 31 46 30  BB2FA36A  AA9541F0
0000004C  42 42 32 46 41 33 36 41  41 41 39 35 34 31 46 30  BB2FA36A  AA9541F0
0000005C  42 42 32 46 41 33 36 41  41 41 39 35 34 58 70 08  BB2FA36A  AA954Xp.
0000006C  74 42 32 46 41 33 36 41  41 41 39 35 34 31 46 30  tB2FA36A  AA9541F0
0000007C  42 42 32 46 41 33 36 41  41 41 39 35 34 31 46 30  BB2FA36A  AA9541F0
0000008C  42 42 32 46 41 33 36 41  41 41 39 35 34 31 46 30  BB2FA36A  AA9541F0
0000009C  42 42 32 46 41 33 36 41  41 41 39 35 34 31 74 1a  BB2FA36A  AA9541t.
000000AC  71 77 05 71 41 33 36 41  41 41 39 35 34 31 46 30  qw.qA36A  AA9541F0
000000BC  42 42 32 46 41 33 36 41  41 41 39 35 34 31 46 30  BB2FA36A  AA9541F0
000000CC  42 42 32 46 41 33 36 41  41 41 39 35 34 31 46 30  BB2FA36A  AA9541F0
000000DC  42 42 32 46 41 33 36 41  41 41 39 35 34 31 46 5c  BB2FA36A  AA9541F\
000000EC  34 25 41 2e 2a 45 5a 32  32 22 56 4f 42 5a 35 5f  4%A.*EZ2  2"VOBZ5_
000000FC  36 2e 57 33 2a 59 57 31  24 26 50 53 41 42 35 30  6.W3*YW1  $&PSAB50
0000010C  11 16 73 12 08 70 36 41  41 41 39 35 34 31 46 30  ..s..p6A  AA9541F0
0000011C  73 6c 00 68 70 33 36 41  41 41 39 35 34 31 46 30  sl.hp36A  AA9541F0
```

encrypted register message

C2 Info Extraction

360 INTERNET SECURITY CENTER

- C&C hostname/IP and port could be used for the purposes of blacklist and botnet tracking
- It's not efficient that running the LDX sample in sandbox and extracting C2 info from captured network traffic
  - Since there are >= 2 C2s hardcode in LDX samples and not all of them are contacted
- Our solution is to statically analyze the sample and directly extract the C2 info
  - Automatically done by program

Code Snippet for Decryption

360 INTERNET SECURITY CENTER

- Only 2 kinds of binary decryption code snippets were found in all LDX samples
  - They had fixed instruction sequences
- 2 YARA rules could be written for the snippets
- Encrypted C2 list could be located by analyzing the data references in decryption code snippets

Snippet 1

```
// sample_md5="2d7492e904cd98016e95696fb50891c7", ELF32:Intel80386:UNIX-SystemV
/*
.text:0804F12F C7 44 24 08 00 02 00 00          mov    dword ptr [esp+8], 200h
.text:0804F137 C7 44 24 04 4C 32 0B 08          mov    dword ptr [esp+4], offset a1_0 ; ",1"
.text:0804F13F C7 04 24 C0 1C 0D 08             mov    dword ptr [esp], offset remotestr
.text:0804F146 E8 05 77 01 00                   call   memmove
.text:0804F14B C7 44 24 04 00 02 00 00          mov    dword ptr [esp+4], 200h
.text:0804F153 C7 04 24 C0 1C 0D 08             mov    dword ptr [esp], offset remotestr
.text:0804F15A E8 95 A2 FF FF                   call   encrypt_code
*/
$instrs_cc_x86_1 = {C7 44 24 08 00 02 00 00 C7 44 24 04 [3] 08 C7 04 24 [3] 08 E8 [4] C7 44 24 04
00 02 00 00 C7 04 24 [3] 08 E8 [3] FF}
```

Snippet 2

```
// sample_md5="d3c2aa4825bb930808ac739d0036e4e5", ELF32:Intel80386:UNIX-SystemV
/*
.text:08050D8C BA 80 85 05 08          mov    edx, offset  remotestr
.text:08050D91 B8 C8 2C 05 08          mov    eax, offset  unk_8052CC8
.text:08050D96 B9 24 00 00 00          mov    ecx, 24h
.text:08050D9B 89 D7                   mov    edi, edx
.text:08050D9D 89 C6                   mov    esi, eax
.text:08050D9F F3 A5                   rep movsd
.text:08050DA1 89 F0                   mov    eax, esi
.text:08050DA3 89 FA                   mov    edx, edi
.text:08050DA5 0F B6 08                movzx  ecx, byte ptr [eax]
.text:08050DA8 88 0A                   mov    [edx], cl
.text:08050DAA 83 C2 01                add    edx, 1
.text:08050DAD 83 C0 01                add    eax, 1
.text:08050DB0 C7 44 24 04 00 02 00 00 mov    dword ptr [esp+4], 200h
.text:08050DB8 C7 04 24 80 85 05 08    mov    dword ptr [esp], offset remotestr
.text:08050DBF E8 1D 9D FF FF          call   encrypt_code
 */
```

$instrs_cc_x86_2 = {00 00 BA [3] 08 B8 [3] 08 b9 ?? 00 00 00 89 d7 89 c6 f3 a5 89 f0 89 fa 0f b6 08 88 0a 83 c2 01 83 c0 01 c7 44 24 04 00 02 00 00 c7 04 24 [3] 08 e8 [3] ff}

C2 Extraction Steps

360 INTERNET SECURITY CENTER

1. Locating the decryption code snippet using YARA
2. Locating the encrypted C2 list by analyzing the data references of the found code snippet
   - 2 Python libraries named *elfesteem* and *miasm2* are used to parse and disassemble the ELF file.
3. Decrypting the encrypted C2 list using the never-changed XOR key

Sample C2 Lists

360 INTERNET SECURITY CENTER

- C2 list has fixed format: a couple of “hostname/IP:port” pairs joined by “|”
- The size of C2 list varies from 2~7

Statistics of The Extracted C2s

360 INTERNET SECURITY CENTER

- 787 unique samples were collected in the past 10 months
- 148 C2 servers were extracted
  - Except for invalid ones, 69 of them are IP addresses
- 98 C2 ports were found
  - 43 of them have values of 28xx
  - Only one port greater than 10,000

The C&C Protocol

360 INTERNET SECURITY CENTER

- A custom protocol running on TCP
- C&C connection is persistently kept after being built
- 6 message types: REGISTER, ATTACK, …
- All messages are prefixed with a fixed format header

```
struct msg_hdr  {
    u32 crc;  // CRC of msg_hdr except for this field
    u32 body_len;  // 0x110 for REGISTER, variable for ATTACK
    // following fields mainly make sense for ATTACK message
    u32 cmd_code; // ATK, DOWNLOAD, UPDATE, …
    u32 tasks;
    u32 atk_time;
    u32 sip_low;  // the begin of fake source IP
    u32 sip_high;
};
```

- Message body is encrypted before being transmitted

The REGISTER message

- The first message send by bot after C&C session being built
  - Also called HELLO, LOGIN, etc.
- LDX REGISTER message is used to tell the server the bot's info of CPU, OS version and net speed
  - And 2 fields copied from sample for verification

```
struct register_msg {
    u32 cpu_spd;
    u32 lan_spd;
    u32 lkm_info;  // the rootkit module
    char os[0x41];  // e.g., "2.6.30-rc5-custom"
    char os_cpu[0x41];  // e.g., "i686"
    char cpu_info[0x41];  // e.g., "1*3392"
    char id[0x21];  // self-produced id string
    u8 magic[0x10];  // copied from sample
    u8 version[0x10];  // copied from sample
};
```

The ATTACK Message

360 INTERNET SECURITY CENTER

- Basically it tells the bot to attack *what* targets using *what* type of attack with *what* arguments
- More than one tasks could be contained
  - One task corresponds to one pair of target IP/port

```
struct task {
    u32 dip;  // target IP
    u16 dport;  // target port
    union {
        char domain[0x102];  // when dns_flood
        char not_used[0x102];  // when syn_flood
    } u;
    u32 atk_type;   // syn_flood, dns_flood, tcp_ack_flood
    u32 use_fake_sip;  // if 0, the attack packet is packed with bot's real IP
    u32 payload_size; // offset=0x110
};
```

About Attacking Thread

360 INTERNET SECURITY CENTER

- It's common that the attack argument of thread number is assigned by C2 server
- In case of LDX, however, it's dynamically calculated by bot itself according to its hardware configuration
  - The maximum value is 8
- Every thread will perform the task(s) repeatedly until the assigned attack time elapses

Fake Source IP Generation Algorithm

360 INTERNET SECURITY CENTER

- 65,535 random DWORDs will be generated when an ATTACK message is received
  - srandom(current_time)/random() are called 65,535 times
- Those DWORDs will be repeatedly used in every task to generate fake source IPs
  - current_fake_sip = sip_low + current_dword % (sip_high - sip_low)

Our Command Tracking System

360 INTERNET SECURITY CENTER

- The aim is to see what commands are distributed by the C2 server
  - e.g., command type, attack types/targets /arguments
- A botnet is uniquely identified by 3-tupe of {bot family, C2 hostname/IP, C2 port}
  - Botnets are classified into families based on their C&C protocols

Some Implementation Details

360 INTERNET SECURITY CENTER

- Detailed C&C protocols are acquired by reverse engineering bot samples
  - Message types/formats/exchange
- Tracker programs are written for each family
- C2 hostname/IP and port are extracted from samples
  - Dynamic/static analysis
- The C2 servers are directly contacted by the corresponding tracker programs to receive their commands

A Sample Tracked Command

360 INTERNET SECURITY CENTER

time	family	C2 hostname	C2 IP	C2 port	cmd type	notes
2015/08/23 21:39:55	ldx	xxxx.yyy.net	xxx.xxx.xxx.xxx	24xx	ddos	syn_flood, target=61.220.xx.xx, port=1935, atk_time=30s, payload_size=888, tasks=2, use_fake_source_ip, sip_low=xx.xx.xx.xx, sip_high=xx.xx.xx.xx

A Summary of Tracked Attacks

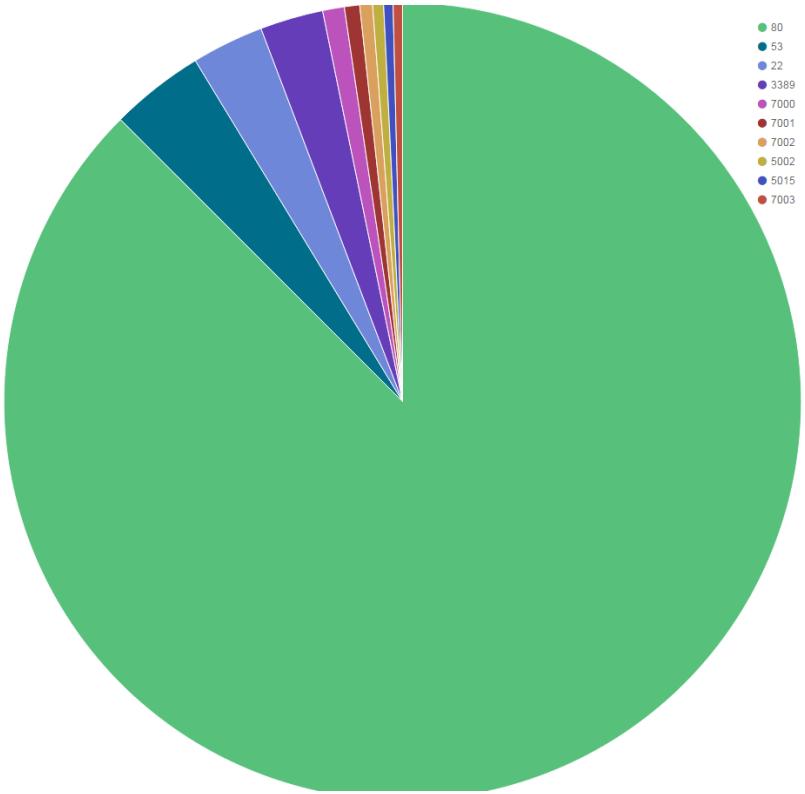
360 INTERNET SECURITY CENTER

- >150M attack commands were received from 213 LDX botnets in the past 10 months, 38,563 targets were detected
  - Those botnets were run on 77 C2 servers over 74 TCP ports
  - 105 active C2 IPs were found
- The most frequently used attack type is syn_flood
  - >95% seen attacks had type of syn_flood

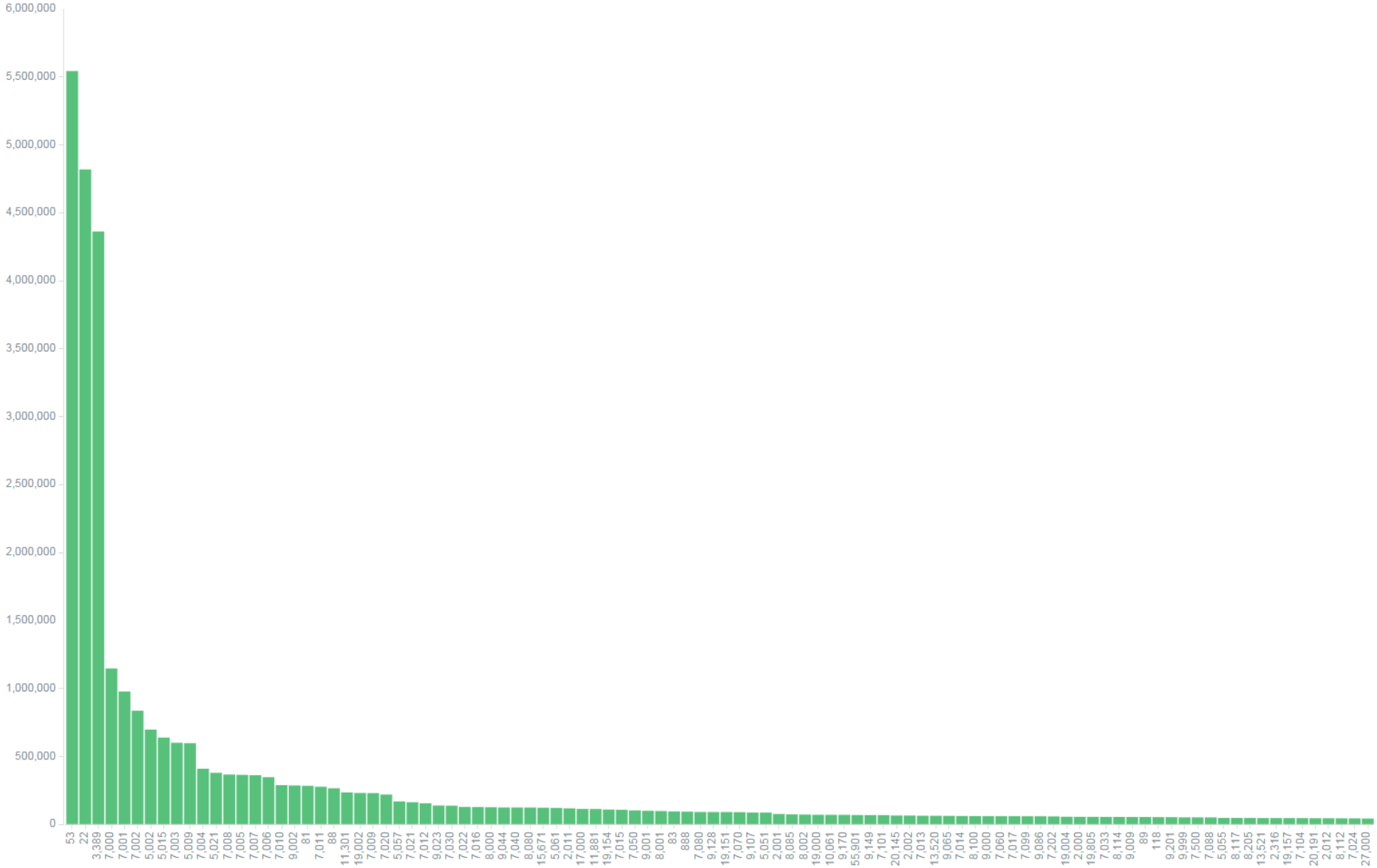
Top 10 Attacked Ports

360 INTERNET SECURITY CENTER

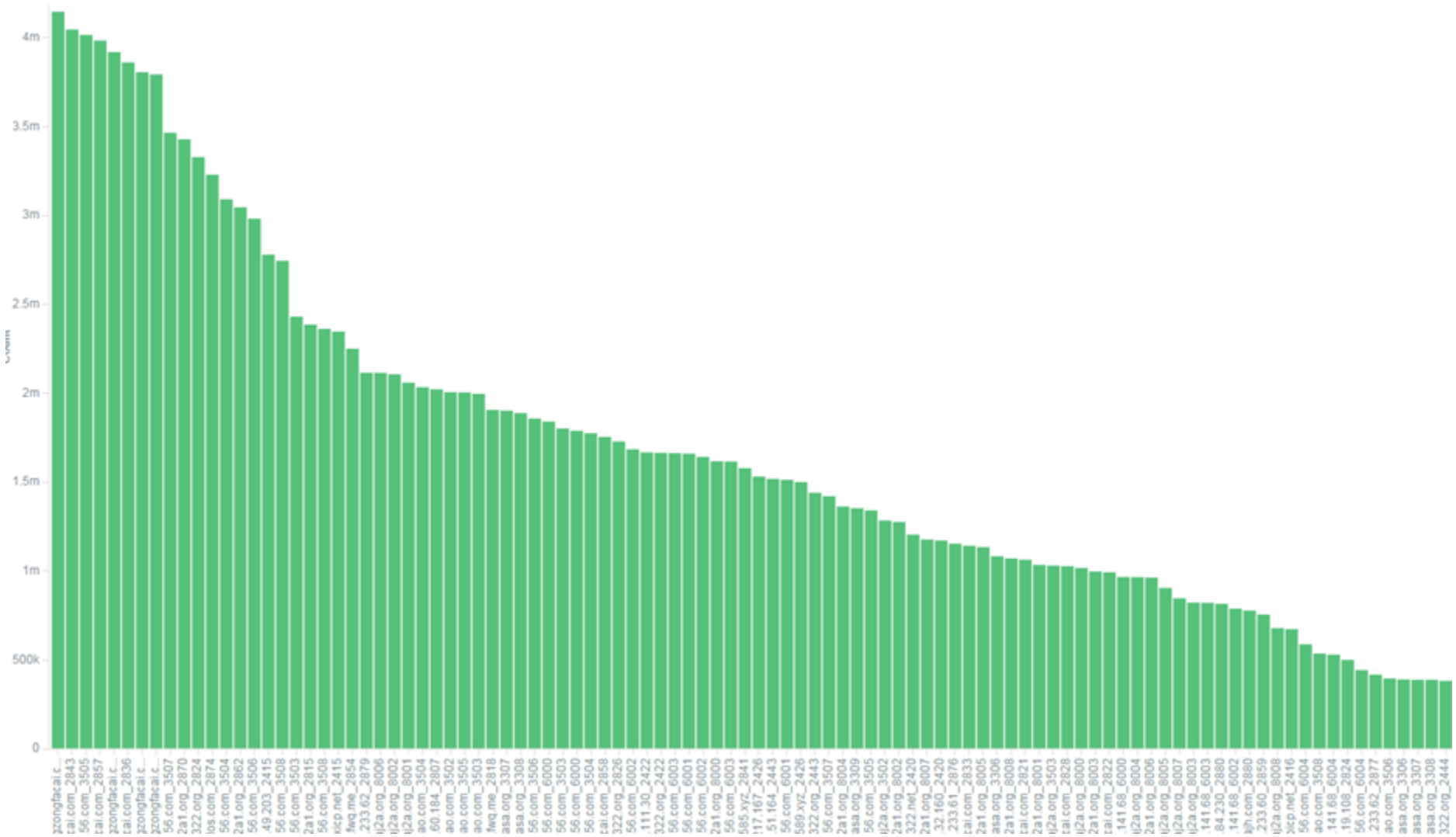
- >95% attacked port is TCP port 80
- syn_flood attacks against DNS were common
- The left ports were mostly used for online games



Top 100 Attacked Ports Except 80



Top 100 Active Botnets



Target GEO Distribution

360 INTERNET SECURITY CENTER

- ~22K in China, ~12K in USA
- 33 target IPs were in POLAND
- Others in FR/HK/…

Targets in Map

360 INTERNET SECURITY CENTER

Who were attacked?

360 INTERNET SECURITY CENTER

- ALIBABA has the largest number of attacked IPs
  - ALIBABA is a cloud computing provider in CHINA
- Amazon/Cloudflare/Google/Microsoft are other target IP owners

Repeated Commands in 1 Attack

360 INTERNET SECURITY CENTER

- When launching attacks, the C2s usually distributed multiple ATTACK commands for each target with short intervals

  # Idx:xxx.xxx.xxx.xxx:yyyy, 20150207200000~20150207230000

  – target=101.71.xx.x1, target_port=7002, atk_type=syn_flood, atk_times: 30, cmd_intervals: [35 38 32 36 36 34 35 35 35 35 36 35 35 35 36 35 35 36 36]

  – target=101.71. xx.x2, target_port=7007, atk_type=syn_flood, atk_times: 30, cmd_intervals: [35 38 32 36 36 34 35 35 35 35 36 35 35 35 36 35 35 36 36]

  – target=101.71. xx.x3, target_port=7021, atk_type=syn_flood, atk_times: 30, cmd_intervals: [35 38 32 36 36 34 35 35 35 35 36 35 35 35 36 35 35 36 36]

  – target=101.71. xx.x4, target_port=9903, atk_type=syn_flood, atk_times: 30, cmd_intervals: [351 351 351 350]

Joint Attacks on Same Targets

360 INTERNET SECURITY CENTER

- It's often seen that multiple LDX botnets jointly attacked the same targets

```
2015/08/23 21:39:55    12[redacted]  2443    syn_flood, target=61.220.[redacted], port=1935, atk_time=30s, payload_size=888, tasks=2, use_fake
2015/08/23 21:39:55    14[redacted]  2420    syn_flood, target=61.220.[redacted], port=1935, atk_time=30s, payload_size=888, tasks=2, use_fake
2015/08/23 21:39:55    18[redacted]  2422    syn_flood, target=61.220.[redacted], port=1935, atk_time=30s, payload_size=888, tasks=2, use_fake
2015/08/23 21:39:55    19[redacted]  2444    syn_flood, target=61.220.[redacted], port=1935, atk_time=30s, payload_size=888, tasks=2, use_fake
2015/08/23 21:39:55    61[redacted]  2415    syn_flood, target=61.220.[redacted], port=1935, atk_time=30s, payload_size=888, tasks=2, use_fake
2015/08/23 21:39:55    jo[redacted]  2443    syn_flood, target=61.220.[redacted], port=1935, atk_time=30s, payload_size=888, tasks=2, use_fake
2015/08/23 21:39:55    17[redacted]  2826    syn_flood, target=61.220.[redacted], port=1935, atk_time=30s, payload_size=888, tasks=2, use_fake
2015/08/23 21:39:55    p[redacted]et      2415    syn_flood, target=61.220.[redacted] port=1935, atk_time=30s, payload_size=888, tasks=2,
2015/08/23 21:39:55    s[redacted]org     2444    syn_flood, target=61.220.[redacted] port=1935, atk_time=30s, payload_size=888, tasks=2,
2015/08/23 21:39:55    s[redacted]  2422    syn_flood, target=61.220.[redacted], port=1935, atk_time=30s, payload_size=888, tasks=2, use_fake
2015/08/23 21:39:55    a[redacted]t       2420    syn_flood, target=61.220.[redacted], port=1935, atk_time=30s, payload_size=888, tasks=2,
2015/08/23 21:40:30    1[redacted]  2443    syn_flood, target=61.220.[redacted], port=1935, atk_time=30s, payload_size=888, tasks=2, use_fake
```

- Presumably those botnets were rented by the same "customer"
  - Or they were run by the same bot master

Different IPs, Same Target?

360 INTERNET SECURITY CENTER

- The C2s often instructed their bots to launch attacks to multiple IPs in the same class C network

  # Idx:xx.xx.xx.xx:yyyy, 20150522010000~20150522120000

  - target=198.41.12x.xx, target_port=80
  - target=198.41.12x.xx, target_port=80

  - target=198.50.23x.xx, target_port=80
  - target=198.50.23x.xx, target_port=80
  - target=198.50.23x.xx, target_port=80
  - target=198.50.23x.xx, target_port=80
  - target=198.50.23x.xx, target_port=80

- Presumably the real targets were the class C networks
  - Or the IDC?

Other Findings

360 INTERNET SECURITY CENTER

- The bot's public IP was always used as sip_low/sip_high
  - sip_low always equaled to sip_high
- All commands had a fixed attack time: 30 seconds
- All dns_flood attacks targeted UDP port 53

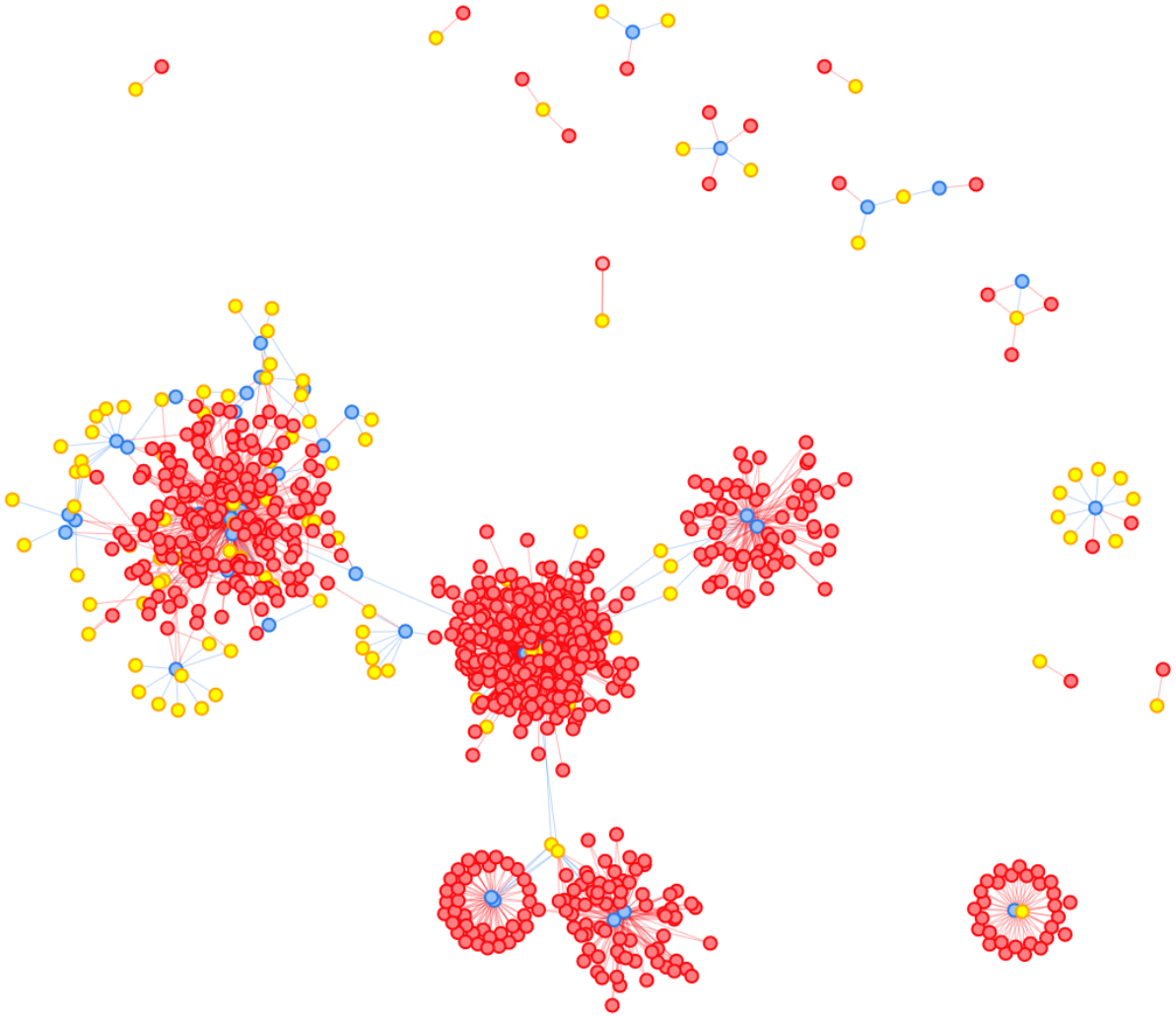
Grouping C2 Servers

360 INTERNET SECURITY CENTER

- It's not easy for the crooks to change their infrastructure frequently, so it's possible to identify them by their IPs
- C2 servers were grouped by following relationships:
  1. Different hostnames were parsed to the same IPs
  2. Multiple C2s were extracted from the same sample

Net of LDX C2 Servers

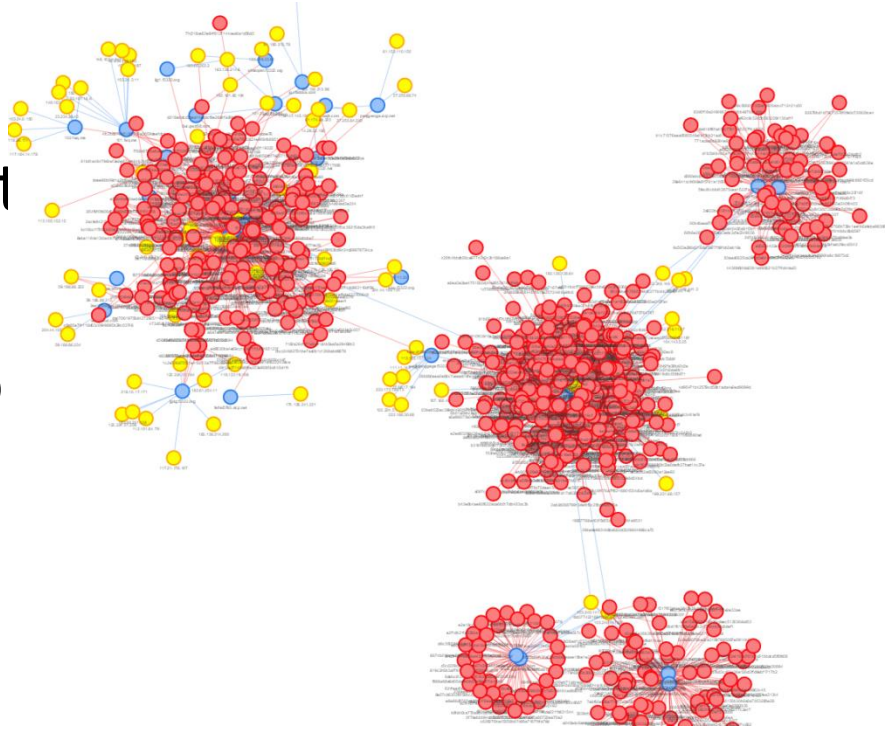
360 INTERNET SECURITY CENTER



The Largest C2 Server Net: N1

360 INTERNET SECURITY CENTER

- There were 81 C2 IPs and 34 hostnames in N1 net
  - Most of LDX botnets we tracked belonged to this net
  - Other families of botnets were found to be related to this net
    - elknot/mayday
- It's thought they were run by a very powerful group of crooks



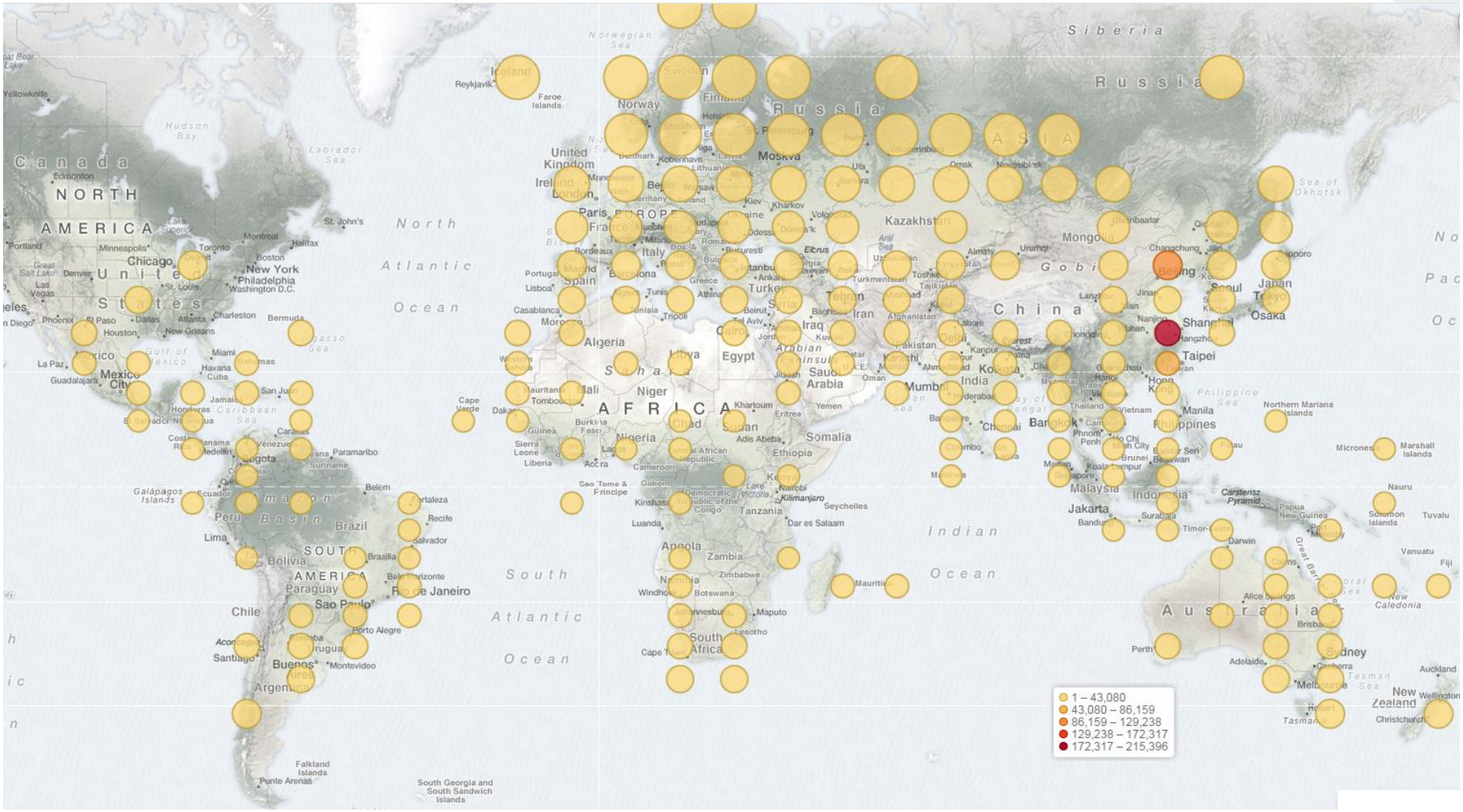
LDX Bots

360 INTERNET SECURITY CENTER

- It's hard to calculate the exact botnet scale
  - The botnets' scale changed from time to time
- Speculations were made based on flow info
- About 470K IPs were found attempting to connect to LDX C2 IPs/ports
  - It's assumed that they are LDX bots
- Most of the IPs are in CHINA

LDX Bots Distribution in Map

360 INTERNET SECURITY CENTER



Acknowledgement

360 INTERNET SECURITY CENTER

- MalwareMustDie!, NPO (@malwaremustdie)
- Andrew Morris (@Andrew___Morris)
- Benkow wokned (@benkow_)
- http://detux.org/

Q&A